

Introduction to Machine Learning (SS 2026)

Programming Project

Author 1

Last name: [TODO: last name]
First name: [TODO: first name]
Matrikel Nr.: [TODO: Matrikel Nr.]

Author 2

Last name: Beneduce
First name: Paul
Matrikel Nr.: 12309055

I. INTRODUCTION

We address a supervised *regression* task: predicting the daily total `power_consumption` of a region from weather observations. Accurately anticipating demand from weather is directly relevant for energy suppliers, who must match generation to load.

The dataset (`powerpredict`) contains 39,997 instances and 65 input features. Each feature describes the weather in one of five fictional cities (Bedrock, Gotham City, New York, Paperopolis, Springfield). Per city there are 11 numeric measurements (temperature and its daily low/high, pressure, humidity, wind speed and direction, short-/long-term rain, snow, cloud cover) and 2 textual weather descriptors (`weather_main`, `weather_description`), giving 55 numeric and 10 categorical columns in total. The target is a single continuous value. The numeric features contain no missing values, so no imputation was required, and they live on very different scales (e.g. pressure vs. rainfall), which motivates the scaling step described below. The target is [TODO: describe target distribution from `eda.ipynb`: e.g. right-skewed, range and mean $\approx$ value]. We evaluate all models with the *Mean Absolute Error* (MAE), the official competition metric (lower is better).

II. IMPLEMENTATION / ML PROCESS

Data preprocessing

Both methods share one preprocessing stage, implemented as a scikit-learn `ColumnTransformer` inside a `Pipeline`. Numeric columns are standardised with `StandardScaler` (zero mean, unit variance). This is essential for a distance-based method such as k-NN: without it, large-range features (e.g. pressure) would dominate the neighbour search and drown out small-range ones (e.g. rainfall). The textual weather descriptors are turned into indicator columns with `OneHotEncoder` (`handle_unknown="ignore"`, so unseen weather categories in the hidden test set do not cause errors). We treated the use of these categorical columns as a design choice and compared both variants (Section III). Wrapping preprocessing and model in a single pipeline guarantees that

the scaler and encoder are fitted only on the training folds, preventing data leakage during cross-validation, and lets the evaluation notebook feed in the raw table directly.

Method 1: k-Nearest Neighbors regression

Method 1 is a k-Nearest-Neighbors regressor. It is an instance-based, non-parametric method: it stores the (scaled) training data and predicts a new point by averaging the targets of its k closest neighbours under the Euclidean (Minkowski, $p = 2$) distance. We use distance weighting, so closer neighbours contribute more strongly to the prediction than distant ones.

k-NN is a natural fit here: the weather-to-power relationship is likely non-linear and locally smooth (similar weather days yield similar consumption), which is exactly the structure k-NN exploits without assuming a global functional form. It also serves as a strong, simple contrast to the linear Method 2. As a non-parametric estimator it is closely related to the non-parametric density estimation covered in the VO [1]; k-NN regression replaces the density estimate with a local average of the target.

Hyperparameters

We selected hyperparameters with an exhaustive grid search using cross-validation, scored by (negative) MAE, so the choice reflects performance on held-out folds rather than the training data. The search grid was the number of neighbours $k \in \{3, 5, 11, 25, 51\}$ and the weighting scheme (*uniform* or *distance*). The best configuration was $k = 51$ with distance weighting. The final submitted model uses the numeric features only (categorical encoding disabled): dropping the high-cardinality one-hot columns kept memory within the JupyterHub limit and did not meaningfully hurt accuracy (Section III).

III. RESULTS

Table I reports the cross-validated MAE of the k-NN model across the hyperparameter grid, and Fig. 1 visualises how the validation MAE changes with k . Larger k with distance weighting gave the lowest error, i.e. $k = 51$ was selected. Table II summarises the final performance against

the dummy baseline (a regressor that always predicts the training-set mean).

TABLE I
CROSS-VALIDATED MAE OF THE K-NN MODEL FOR EACH
HYPERPARAMETER CONFIGURATION (LOWER IS BETTER).

k (neighbours)	weights	CV MAE
3	uniform	[TODO:]
3	distance	[TODO:]
5	uniform	[TODO:]
5	distance	[TODO:]
11	uniform	[TODO:]
11	distance	[TODO:]
25	uniform	[TODO:]
25	distance	[TODO:]
51	uniform	[TODO:]
51	distance	3509.56

[TODO: insert figures/knn_val_mae_vs_k.png]

Fig. 1. Validation MAE of the k-NN regressor as a function of the number of neighbours k , for uniform and distance weighting.

TABLE II
FINAL MAE ON THE TRAINING, VALIDATION AND (HIDDEN) TEST SETS.
LOWER IS BETTER.

Model	Train MAE	Valid. MAE	Test MAE
Dummy (mean)	[TODO:]	[TODO:]	[TODO:]
k-NN (Method 1)	15.69	3509.56	2438.15
Ridge (Method 2)	[TODO:]	[TODO:]	[TODO:]

IV. DISCUSSION

The k-NN model clearly beats the dummy baseline on unseen data (test MAE 2438 vs. dummy [TODO: value]), confirming that the weather features carry real predictive signal. The very low *training* MAE (15.69) is expected and not a sign of quality: with distance weighting, a query that coincides with a stored training point receives essentially that point’s own target (weight $\rightarrow \infty$ at distance 0), so the model reproduces the training data almost exactly. The gap to the validation/test MAE (≈ 2400 – 3500) is the true generalisation error and the number that matters.

Interestingly, the test MAE (2438) is lower than the cross-validation estimate (3509). This is plausible because each CV fold trains on only two thirds of the data, whereas the final model is refit on the full training set, and because the hidden test set may be slightly easier; in any case it indicates the model is not overfitting to the public data.

Two things we tried but dropped: (i) one-hot encoding the weather-text columns inflated the feature space and, together with the memory footprint of k-NN, exhausted the JupyterHub RAM (the training process was killed); the gain in accuracy was negligible [TODO: cite the with/without-encoding numbers from training.ipynb, cell 4], so we kept the leaner numeric-only model. (ii) Running the grid search in parallel (`n_jobs=-1`) replicated the full dataset across

worker processes and triggered the same out-of-memory failure, so the search is run sequentially.

The best k (51) sat at the upper edge of our grid, which suggests even larger neighbourhoods might reduce error further; extending the grid (e.g. $k \in \{75, 101\}$) is the obvious next step. Beyond that, dimensionality reduction such as PCA [2] could counter the curse of dimensionality that hurts distance-based methods, and feature selection or an alternative distance metric could sharpen the neighbour search.

V. CONCLUSION

Our final k-NN regressor achieves a test-set MAE of **2438.15**, substantially better than the dummy baseline of [TODO: value]. We did not tune against the test set; hyperparameters were fixed purely on cross-validated training performance.

The main takeaway is that a simple, non-parametric method with careful preprocessing goes a long way on this problem: standardising the weather features and letting k-NN average over locally similar days already yields a large improvement over the mean predictor. The exercise also highlighted the practical trade-offs of instance-based learning: k-NN’s memory cost forced us to drop the categorical features and run the search sequentially, and its tendency to memorise the training data makes the training error misleading. A regularised linear model (Method 2) offers a complementary, low-memory perspective, and comparing the two clarifies when local averaging beats a global linear fit for weather-driven demand prediction.

REFERENCES

- [1] Introduction to Machine Learning (SS 2026), Lecture 10: *Nonparametric Density Estimation*. University of Innsbruck.
- [2] Introduction to Machine Learning (SS 2026), Lecture 07: *Dimensionality Reduction by PCA/SVD*. University of Innsbruck.